

**Girijananda Chowdhury Institute of Management
and Technology**



PROJECT REPORT

ON

*“A GPU-Inspired Parallel Accelerator for K-Means
Image Segmentation on FPGA”*

Submitted in the partial fulfillment of requirement for the Degree of

Bachelor of Technology

in

DEPARTMENT OF

ELECTRONICS AND COMMUNICATIONS ENGINEERING



**ASSAM SCIENCE AND TECHNOLOGY
UNIVERSITY, GUWAHATI**

SUBMITTED BY:

Name: Akarshan Ghosh 220310006001
B.Tech ECE, 8th Semester

PROJECT GUIDE:

Dr. Basab Das
Assistant Professor
Dept. of ECE, GIMT

**Girijananda Chowdhury Institute of Management
and Technology**



SESSION: 2025–2026

DEPARTMENT OF
ELECTRONICS AND COMMUNICATIONS ENGINEERING

CERTIFICATE

This is to certify that **Akarshan Ghosh**, student of B.Tech 4th Year, 8th Semester, has completed the project entitled “**A GPU-Inspired Parallel Accelerator for K-Means Image Segmentation on FPGA**” during the academic session 2025–2026 (January – May) under my supervision and guidance. I approve the project for submission as required for partial fulfillment for the completion of the Bachelor of Technology Degree.

PROJECT GUIDE:

Dr. Basab Das
Assistant Professor
ECE, GIMT

Dr. Anindita Bora
Associate Professor &
Head of the Department
ECE, GIMT

Prof. S. Robert Ravi
Principal, GIMT

DECLARATION

We hereby declare that this project work entitled “**A GPU-Inspired Parallel Accelerator for K-Means Image Segmentation on FPGA**” was carried out by us under the guidance and supervision of Dr. Basab Das, Girijananda Chowdhury Institute of Management and Technology. This project is submitted to the Department of Electronics and Communication Engineering (ECE) during the academic year 2025–2026. The work has never been produced before any authority except Assam Science & Technology University for evaluation.

Name Of Project Members

Roll No.

Signature

Akarshan Ghosh

220310006001

Acknowledgement

We are grateful to **Mrs. Anindita Bora**, Head of the Department of Electronics and Communication Engineering, GIMT, Guwahati, for her cheerful encouragement and valuable suggestions throughout the course of this project.

We wish to express our heartfelt thanks to our guide **Mr. Basab Das**, Assistant Professor, Department of **Electronics and Communication Engineering**, GIMT, Guwahati, for his invaluable guidance, constant supervision, and for providing the necessary information and support that enabled us to successfully complete this project.

Our special thanks and appreciation also go to all the faculty members and technical staff of the ECE Department, GIMT, Guwahati, for their encouragement and kind cooperation, which greatly helped us in the completion of this project.

Lastly, we express our sincere gratitude to our parents and all our friends who helped us in one way or another.

Name Of Project Members

Roll No.

Akarshan Ghosh

220310006001

ABSTRACT

This project presents the complete design, implementation, and verification of a GPU-inspired parallel hardware accelerator for K-Means image segmentation, designed entirely in Verilog HDL and simulated using Xilinx Vivado 2025. The accelerator operates on 32×32 RGB images, processing 1024 pixels across 20 K-Means iterations to produce a segmented output with four distinct colour clusters.

The core novelty of the design lies in its parallel computation principle: four distance computation units (`dist_unit.v`) fire simultaneously in a single clock cycle—one per centroid—mirroring the GPU thread-parallel model in FPGA silicon. A two-level tournament comparator (`min_finder_k4.v`) finds the winning centroid in $\log_2 K$ comparator stages rather than a serial chain, minimising critical path depth. A seven-state Finite State Machine (`fsm_control.v`) orchestrates the entire pipeline through IDLE, LOAD, COMPUTE, UPDATE, WRITE, CLEAR, and DONE states.

Python serves exclusively as a data preparation and validation tool. An image is resized, flattened, and exported as hex memory files using K-Means++ initialisation to guarantee spread initial centroids and prevent cluster death. Vivado loads these files at simulation start via the `$readmemh` system task, ensuring the hardware operates on authentic image data without any manual intervention.

All seven Verilog modules were verified individually through dedicated testbenches, achieving 39 out of 39 test cases passed with zero failures. Full-system simulation confirmed correct operation across all 20 iterations, completing 20,480 pixel operations in approximately 206 microseconds at 100 MHz. A Python validation script confirmed bit-identical results between hardware and software (total L1 centroid delta = 0), providing the strongest possible functional verification.

Comparison with the most closely related published work (Badawi and Bilal, 2019) demonstrates that this design achieves exact mean computation via integer division rather than approximate weighted averaging, and is implemented as fully hand-written RTL Verilog rather than auto-generated HLS code, offering complete timing control and exact functional equivalence with the reference model.

Keywords: K-Means clustering, FPGA, Verilog HDL, hardware accelerator, image segmentation, parallel computing, Xilinx Vivado, FSM, RGB squared Euclidean distance, K-Means++, tournament comparator, RTL design.

TABLE OF CONTENTS

Contents

1	Introduction	1
1.1	Background	1
1.2	The Computational Challenge	1
1.3	Motivation	2
1.4	Objectives	2
1.5	Project Overview and Scope	3
1.6	Organisation of the Report	3
2	Literature Review	3
2.1	Overview	4
2.2	FPGA-Based K-Means Hardware Accelerators	4
2.2.1	Hussain et al. (2011)	4
2.2.2	Kutty et al. (2013)	4
2.2.3	Badawi and Bilal (2019) — Primary Related Work	4
2.2.4	KPynq (2019)	5
2.2.5	FPGA Image Processing Acceleration (2021)	5
2.3	GPU-Parallel K-Means	5
2.3.1	Zechner and Granitzer (2009)	5
2.3.2	IEEE Big Data (2015)	6
2.4	K-Means++ Initialisation Theory	6
2.5	HW/SW Co-Design for K-Means	6
2.6	Summary	7
3	K-Means Algorithm and Mathematical Model	7
3.1	Overview	8
3.2	The K-Means Clustering Algorithm	8
3.3	Squared Euclidean Distance	8
3.4	Signed Subtraction and Unsigned Wrap-Around	9
3.5	Tournament Comparator — argmin	9
3.6	Centroid Update — Integer Mean	10
3.7	K-Means++ Initialisation	10
3.8	Empty Cluster Guard	11
4	System Architecture	11
4.1	High-Level Workflow	12
4.2	Hardware Pipeline Data Flow	13

4.3	Full RTL Schematic	14
4.4	Write-Back Multiplexer	15
5	Module-Level Design	15
5.1	Overview	16
5.2	dist_unit.v — Distance Computation Unit	16
5.2.1	Purpose and Behaviour	16
5.2.2	Port Description	16
5.2.3	Internal Three-Stage Pipeline	17
5.3	min_finder_k4.v — Tournament Comparator	17
5.3.1	Purpose and Behaviour	17
5.3.2	Tournament Tree Implementation	17
5.4	centroid_regs.v — Centroid Register File	18
5.4.1	Purpose and Behaviour	18
5.4.2	init_load Port — Critical Design Decision	18
5.5	accumulator.v — Sum Banks and Centroid Update	18
5.5.1	Purpose and Behaviour	18
5.6	pixel_mem.v — Image Block RAM	19
5.6.1	Purpose and Behaviour	19
5.7	fsm_control.v — Seven-State FSM	20
5.7.1	Purpose and Behaviour	20
5.7.2	Key Internal Registers	21
5.8	kmeans_top.v — Top-Level Integration Module	22
6	FSM Control Flow	23
6.1	State Transition Logic	24
6.2	The Two Nested Hardware Loops	24
6.3	Critical Timing Constraints	24
7	Hardware Bugs and Fixes	25
7.1	Overview	26
7.2	Bug 1 — Unsigned Subtraction Wrap (dist_unit.v)	26
7.3	Bug 2 — Start Pulse Missed by FSM (fsm_control.v)	27
7.4	Bug 3 — FSM Reset Overwrote Initial Centroids	27
7.5	Bug 4 — Cluster Death Cascade	28
7.6	Bug 5 — Accumulator Not Cleared Between Iterations	28
7.7	Bug 6 — UPDATE/WRITE Timing Race	29
8	Simulation Results and Verification	29
8.1	Unit Test Results	30
8.2	Full System Simulation	30

8.3	Python vs Hardware Validation	31
8.4	Visual Validation Results	32
9	Novelty and Comparison with Related Work	32
9.1	Overview	33
9.2	Detailed Comparison Table	33
9.3	Key Contributions of This Project	34
9.3.1	Contribution 1 — Bit-Identical Hardware Verification	34
9.3.2	Contribution 2 — Hand-Written RTL with Explicit Timing Control .	35
9.3.3	Contribution 3 — K-Means++ in the Hardware Flow	35
9.3.4	Contribution 4 — Empty Cluster Survival Guard	35
10	Conclusion and Future Work	36
10.1	Conclusion	37
10.2	Future Work	37

LIST OF FIGURES

S.No	Figure Name	Page No.
1	Block Diagram	13
2	Detailed hardware data flow: centroid_regs, parallel dist_units, min_finder_k4	14
3	Detailed hardware architecture of the K-Means accelerato	14
4	Full RTL schematic generated by Vivado (12 cells, 337 nets)	15
5	Zoomed RTL schematic of accumulator module	19
6	Zoomed RTL schematic of pixel_mem module	20
7	Zoomed RTL schematic of fsm_control module	21
8	FSM control flow of the K-Means hardware accelerator,	22
9	Zoomed RTL schematic of centroid_regs and dist_units	23
10	Full system Vivado simulation waveform (done=1 at 206 μ s)	31
11	Validation result for Test Image 1	32
12	Validation result for Test Image 2	32

LIST OF TABLES

S.No	Table Name	Page No.
1	Summary of all seven Verilog modules and verification results	16
2	Bit-width analysis for each internal signal of dist_unit.v	9
3	FSM state descriptions and asserted signals	20
4	Unit test results for all five individually tested modules	30
5	Full system simulation parameters and results	30
6	Final centroid comparison between Python reference and hardware output	31
7	Detailed comparison with related FPGA K-Means hardware designs	34
8	Summary of all six hardware bugs, root causes, and fixes	26

Chapter 1: Introduction

1.1 Background

Image segmentation is one of the most fundamental and widely studied tasks in computer vision and image processing. It refers to the process of partitioning a digital image into multiple segments or regions, where each segment corresponds to a group of pixels that share a common visual characteristic such as colour, intensity, or texture. Accurate and efficient image segmentation is essential for higher-level operations such as object detection, scene understanding, medical diagnosis, and autonomous navigation.

Among the many algorithms available for image segmentation, the K-Means clustering algorithm occupies a prominent position due to its conceptual simplicity, predictable convergence behaviour, and effectiveness on colour-based segmentation tasks. Given an image of N pixels and a desired number of colour clusters K , K-Means iteratively assigns each pixel to the nearest cluster centroid and updates the centroids to reflect the mean colour of assigned pixels. The algorithm converges when centroid positions stabilise between iterations.

1.2 The Computational Challenge

Despite its algorithmic simplicity, K-Means imposes a significant computational burden when applied to real-time image streams. For a 32×32 pixel RGB image with $K = 4$ clusters and 20 iterations, the accelerator must perform:

$$1024 \text{ pixels} \times 4 \text{ centroids} \times 20 \text{ iterations} = 81,920 \text{ distance computations}$$

Each distance computation involves three subtractions, three multiplications, and two additions. A conventional CPU executes these sequentially, one centroid at a time, resulting in high latency that precludes real-time operation in resource-constrained embedded systems.

Graphics Processing Units (GPUs) address this bottleneck by executing thousands of threads in parallel, but consume hundreds of watts of power and require large, expensive cooling systems. They are unsuitable for deployment inside camera modules, drones, or industrial sensors.

Field Programmable Gate Arrays (FPGAs) offer a compelling middle ground. They can implement custom parallel architectures that compute all four centroid distances in a single clock cycle—equivalent to four simultaneous GPU threads—while consuming only

milliwatts of power and fitting into compact embedded form factors.

1.3 Motivation

This project is motivated by three specific gaps observed in the existing literature on FPGA-based K-Means accelerators:

1. **Approximate centroid updates:** Most published designs (including the closest related work, Badawi and Bilal, 2019) replace exact division in the centroid update step with a weighted average approximation to avoid expensive divider hardware. This introduces systematic bias and prevents validation against an exact software reference.
2. **Auto-generated HDL:** Many recent designs use High-Level Synthesis (HLS) tools such as Simulink HDL Coder or Xilinx Vivado HLS to auto-generate Verilog or VHDL from block diagrams. While this reduces design time, it hides timing bugs and prevents direct RTL-level control over pipeline stages, register allocation, and state machine transitions.
3. **No cluster death protection:** Random centroid initialisation can place two or more centroids at similar colours, causing one to attract zero pixels and collapse to a degenerate value. No published FPGA design explicitly addresses this issue in the hardware preparation pipeline.

This project addresses all three gaps simultaneously.

1.4 Objectives

The specific objectives of this project are:

1. To design a complete parallel K-Means hardware accelerator using hand-written Verilog HDL.
2. To implement parallel distance calculation: all four centroids computed simultaneously in a single clock cycle using four independent combinational distance units.
3. To implement exact integer mean computation in the accumulator module for bit-accurate centroid updates.
4. To design a seven-state FSM that controls the entire pipeline with explicit timing control over every enable signal.
5. To implement K-Means++ centroid initialisation in Python to provably prevent cluster death in hardware.
6. To verify the complete system by achieving bit-identical output between hardware and a Python reference model (L1 delta = 0).
7. To identify, root-cause, and fix all hardware bugs encountered during Vivado simulation,

documenting the engineering process.

1.5 Project Overview and Scope

The complete accelerator consists of seven Verilog modules: `dist_unit.v`, `min_finder_k4.v`, `centroid_regs.v`, `accumulator.v`, `pixel_mem.v`, `fsm_control.v`, and `kmeans_top.v`. Python scripts handle image preparation (`export_mem.py`) and validation (`validate_kmeans.py`) but perform no clustering logic.

The design targets a 32×32 pixel RGB image (1024 pixels, 4 clusters, 20 iterations) and is simulated at 100 MHz using Xilinx Vivado 2025. The complete source code, testbenches, and Python scripts are publicly available at: <https://github.com/AkarshanGhosh/kmeans-hardware-accelerator>.

1.6 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 reviews relevant prior work on FPGA-based K-Means accelerators, GPU-parallel clustering, and K-Means++ initialisation. Chapter 3 describes the K-Means algorithm and all mathematical models used in the hardware design. Chapter 4 presents the complete system architecture and hardware data flow. Chapter 5 provides a detailed description of all seven Verilog modules. Chapter 6 explains the FSM control flow and state transition logic. Chapter 7 documents all six hardware bugs encountered and the fixes applied. Chapter 8 presents simulation results, unit test outcomes, and Python-vs-hardware validation. Chapter 9 analyses the novelty of this project and compares it against related published designs. Chapter 10 concludes the report and identifies directions for future work.

Chapter 2: Literature Review

2.1 Overview

Hardware acceleration of K-Means clustering has been an active area of research for over two decades, driven by the growing demand for real-time image processing in embedded systems. This chapter surveys the most relevant published works across four categories: FPGA-based K-Means accelerators, GPU-parallel K-Means, K-Means++ initialization theory, and FPGA image processing methodology.

2.2 FPGA-Based K-Means Hardware Accelerators

2.2.1 Hussain et al. (2011)

Hussain, Benkrid, Seker, and Erdogan [2] described one of the earliest high-speed configurable FPGA implementations of K-Means, targeting bioinformatics applications on the Xilinx Virtex-IV platform. Their design achieves 126 MHz and uses multiple parallel distance calculation units arranged in a deeply pipelined architecture. However, the design requires external DRAM for the centroid update feedback loop: after each pass over the dataset, intermediate sums are written to and read from off-chip memory. This external memory bottleneck means the effective throughput is far lower than the clock speed suggests. The design also does not address the real-time image segmentation use case and lacks a software validation framework.

2.2.2 Kutty et al. (2013)

Kutty, Boussaid, and Amira [3] presented a fully pipelined K-Means architecture on the Xilinx Virtex-VI achieving an impressive 400 MHz clock frequency. However, as Badawi and Bilal subsequently noted, the claimed speed figure excludes the feedback loop latency from external memory accesses. The actual effective throughput for a complete K-Means pass including centroid update is significantly lower. This design represents a cautionary example of speed claims that are not representative of end-to-end operation.

2.2.3 Badawi and Bilal (2019) — Primary Related Work

The work of Badawi and Bilal [1] is the most directly comparable to this project. They developed a Simulink HLS-based online K-Means hardware accelerator for colour image segmentation on the Xilinx Zynq-7000 SoC. Their design runs at 55 MHz and processes Full HD (1920×1080) video at 26.5 fps while consuming less than 90 mW of dynamic power.

A key innovation in their work is the replacement of the exact division operation in

the centroid update step with a weighted average:

$$\hat{C}_i = C_i \cdot \alpha + \mathbf{p} \cdot (1 - \alpha), \quad \alpha \approx 0.999$$

This eliminates the need for a hardware divider, significantly reducing logic resource utilisation. The weighted average approximates the true mean as the number of processed pixels increases, reaching near-equivalence after fewer than 10 lines of a VGA frame. Their RMSE compared to an offline Matlab K-Means reference is in the range of 4 to 7 across various test video sequences.

The design uses YCbCr colour space (converted from RGB) rather than raw RGB, arguing that decoupling luminance from chrominance improves clustering quality. It also supports switchable L1 and L2 distance norms under software control.

This is the richest related work and serves as the primary benchmark for comparison in this project. The key differences—exact vs. approximate update, hand-written RTL vs. HLS-generated code, and K-Means++ vs. random initialisation—are discussed in detail in Chapter 9.

2.2.4 KPynq (2019)

The KPynq work [4] demonstrated a triangle-inequality-based K-Means accelerator on the Xilinx Zynq SoC using the Pynq framework. By exploiting the triangle inequality to prune unnecessary distance computations, they achieved $4.2\times$ speedup over an ARM Cortex-A9 CPU while consuming $218\times$ less energy. This work validates the viability of FPGA-based K-Means for energy-efficient processing.

2.2.5 FPGA Image Processing Acceleration (2021)

The work published in Sensors (MDPI) in 2021 [5] demonstrated FPGA-based acceleration of image processing using 16 parallel IPPro processing cores, achieving $8\times$ speedup over an ARM Cortex-A7 CPU and $57\times$ better power efficiency. The paper specifically discusses K-Means as a target workload, confirming that FPGA parallelism provides dramatic power advantages over software implementations on embedded ARM processors.

2.3 GPU-Parallel K-Means

2.3.1 Zechner and Granitzer (2009)

Zechner and Granitzer [6] were among the first to implement K-Means clustering on modern CUDA-capable GPUs. Their implementation assigns each pixel to a separate CUDA thread during the assignment step, so all N distances to K centroids are computed in parallel across the entire dataset. The speedup over sequential CPU implementation was

approximately $30\times$ on large datasets. The GPU parallel-thread model directly inspired the four-parallel-unit architecture in this project: instead of thousands of threads on a GPU, we implement exactly four parallel distance units on an FPGA—one per centroid—which is sufficient for $K = 4$ and consumes far less power.

2.3.2 IEEE Big Data (2015)

The work of [8] benchmarked parallel K-Means implementations across multiple GPU architectures. The study confirmed that the assignment step (distance computation and argmin) dominates the total computation time, accounting for over 80% of runtime. This finding directly motivates the parallelisation strategy in this project: by parallelising the distance computation step through four simultaneous combinational units, the dominant cost of K-Means is addressed at the hardware level.

2.4 K-Means++ Initialisation Theory

Arthur and Vassilvitskii [9] introduced the K-Means++ seeding algorithm in 2007, establishing it as the standard initialisation method for K-Means in practice. The algorithm selects the first centroid uniformly at random and each subsequent centroid with probability proportional to the squared distance from the nearest already-selected centroid. This simple modification provides a $O(\log K)$ approximation guarantee on the final clustering objective value and dramatically reduces the incidence of poor convergence caused by unlucky random initialisation.

The cluster death problem—where a centroid collapses to zero assigned pixels and remains degenerate for all subsequent iterations—is a direct consequence of poor initialisation. K-Means++ provably prevents this in expectation by spreading the initial centroids across the data space. No prior FPGA K-Means design in the surveyed literature integrates K-Means++ into the hardware preparation pipeline.

2.5 HW/SW Co-Design for K-Means

The hardware-software co-design study of [7] systematically compared FPGA, GPU, and multi-core CPU implementations of K-Means across a range of dataset sizes. The study concluded that FPGAs offer the best performance-per-watt for datasets of moderate size (up to a few million points), while GPUs become more competitive only at very large scale. For image processing applications where real-time latency and power budget are the primary constraints, FPGA implementations are the recommended choice. This finding directly supports the architectural decision in this project.

2.6 Summary

The reviewed literature establishes the following design principles that inform this project:

- Parallel distance units are the most effective hardware optimisation for K-Means acceleration.
- On-chip centroid storage eliminates the external memory bottleneck that limits throughput in several prior designs.
- Exact centroid computation (division) is achievable in hardware and enables rigorous validation against software references.
- K-Means++ initialisation should be standard practice to ensure stable convergence.

Chapter 3: K-Means Algorithm and Mathematical Model

3.1 Overview

This chapter presents the complete mathematical model underlying the hardware accelerator. Each equation is directly translated into a Verilog hardware construct, and the mapping between mathematical operations and specific modules is explicitly identified.

3.2 The K-Means Clustering Algorithm

Given an image with N pixels, each represented as a three-dimensional RGB vector $\mathbf{p}_i = (R_i, G_i, B_i)$ where $R_i, G_i, B_i \in [0, 255]$, K-Means clustering partitions the pixels into K groups called clusters.

Each cluster $k \in \{0, 1, \dots, K-1\}$ is characterised by a centroid $\mathbf{c}_k = (R_k, G_k, B_k)$ representing the mean colour of all pixels assigned to that cluster. The algorithm minimises the within-cluster sum of squared distances:

$$J = \sum_{k=0}^{K-1} \sum_{i \in S_k} d(\mathbf{p}_i, \mathbf{c}_k) \quad (1)$$

where S_k is the set of pixels assigned to cluster k and d is the distance metric.

The algorithm proceeds in four steps:

1. **Initialisation:** Select K initial centroids using K-Means++.
2. **Assignment:** Assign each pixel to the nearest centroid.
3. **Update:** Recompute each centroid as the mean of its assigned pixels.
4. **Repeat:** Return to step 2; continue for 20 iterations.

3.3 Squared Euclidean Distance

Hardware module: `dist_unit.v`

The distance between pixel $\mathbf{p}$ and centroid $\mathbf{c}_k$ is the squared Euclidean distance:

$$d(\mathbf{p}, \mathbf{c}_k) = (R_p - R_k)^2 + (G_p - G_k)^2 + (B_p - B_k)^2 \quad (2)$$

Why squared (no square root): The assignment step only requires knowing which centroid is *nearest*, not the actual distance value. Since the square root function is strictly

monotonically increasing:

$$\sqrt{a} < \sqrt{b} \iff a < b \quad \forall a, b \geq 0$$

the ordering of distances is preserved after squaring, so the square root is unnecessary. Removing it eliminates a large combinational circuit from the critical path.

Bit-width analysis: Table 3 shows the bit-width at each stage of the computation.

Table 3: Bit-width analysis for each internal signal of `dist_unit.v`.

Signal	Expression	Width	Max value
<code>pixel_r, cent_r</code>	Raw 8-bit unsigned	8 bits	255
<code>diff_r</code>	<code>pixel_r - cent_r</code> (signed)	9 bits	-255 to +255
<code>sq_r</code>	<code>diff_r × diff_r</code>	18 bits	65,025
<code>dist_sq</code>	<code>sq_r + sq_g + sq_b</code>	18 bits	195,075

The worst-case distance is $3 \times 255^2 = 195,075 < 2^{18} = 262,144$, confirming that 18 bits is sufficient for all distance wires.

3.4 Signed Subtraction and Unsigned Wrap-Around

A critical hardware detail is the signed expansion before subtraction. In 8-bit unsigned arithmetic:

$$100 - 200 = \underbrace{100 - 200 + 256}_{\text{unsigned wrap}} = 156$$

giving a square of $156^2 = 24,336$ instead of the correct $(-100)^2 = 10,000$.

The fix is to expand both operands to 9-bit signed before subtracting using zero-padding to preserve the unsigned magnitude as a positive signed value:

$$\{1'b0, \text{pixel_r}\} - \{1'b0, \text{cent_r}\} \quad (9\text{-bit signed subtraction})$$

3.5 Tournament Comparator — `argmin`

Hardware module: `min_finder_k4.v`

After four distances d_0, d_1, d_2, d_3 are computed in parallel, the nearest centroid is determined by:

$$\text{cluster_id} = \arg \min_{k \in \{0,1,2,3\}} d_k$$

The hardware implements this as a two-level binary tournament tree:

$$\text{Level 1A : } \text{winner_A} = \min(d_0, d_1), \quad \text{id_A} = \arg \min(d_0, d_1) \quad (3)$$

$$\text{Level 1B : } \text{winner_B} = \min(d_2, d_3), \quad \text{id_B} = \arg \min(d_2, d_3) \quad (4)$$

$$\text{Level 2 : } \text{cluster_id} = \arg \min(\text{winner_A}, \text{winner_B}) \quad (5)$$

Levels 1A and 1B execute simultaneously in the same clock cycle. The critical path depth is two comparator stages, versus three for a serial chain (d_0 vs d_1 , winner vs d_2 , winner vs d_3).

3.6 Centroid Update — Integer Mean

Hardware module: `accumulator.v`

After all $N = 1024$ pixels are processed, the new centroid for cluster k is:

$$\hat{C}_k = \frac{1}{n_k} \sum_{i \in S_k} \mathbf{p}_i \quad (6)$$

where n_k is the number of pixels assigned to cluster k . The hardware accumulates the sums during the COMPUTE state:

$$\text{sum_r}[k] += R_i, \quad \text{sum_g}[k] += G_i, \quad \text{sum_b}[k] += B_i, \quad \text{count}[k] += 1$$

After the full pass, the UPDATE state computes the integer division:

$$\hat{C}_{k,R} = \left\lfloor \frac{\text{sum_r}[k]}{\text{count}[k]} \right\rfloor$$

The floor (truncation) of integer division matches Python's `int()` function, ensuring bit-identical results between hardware and software.

3.7 K-Means++ Initialisation

Python module: `export_mem.py`

The first centroid $\mathbf{c}_0$ is chosen uniformly at random. Each subsequent centroid $\mathbf{c}_j$ ($j \geq 1$) is selected according to the probability distribution:

$$P(\mathbf{p}) = \frac{D(\mathbf{p})^2}{\sum_{\mathbf{q}} D(\mathbf{q})^2} \quad (7)$$

where $D(\mathbf{p}) = \min_{i < j} d(\mathbf{p}, \mathbf{c}_i)$ is the minimum squared distance from pixel $\mathbf{p}$ to any already-selected centroid. This biases selection toward pixels far from existing centroids, spreading the initial centroids across the colour space.

3.8 Empty Cluster Guard

Hardware module: `accumulator.v`

When $n_k = 0$, the division $0/0$ is undefined. The guard replaces the new centroid with the previous centroid value:

$$\hat{C}_k = \begin{cases} \left\lfloor \frac{\text{sum}[k]}{n_k} \right\rfloor & \text{if } n_k > 0 \\ C_k^{\text{prev}} & \text{if } n_k = 0 \end{cases} \quad (8)$$

The previous centroid values are latched into internal registers at the moment `update_en` fires, before any write to `centroid_regs` occurs, preventing a read-during-write timing hazard.

Chapter 4: System Architecture

4.1 High-Level Workflow

The complete workflow of the K-Means hardware accelerator is divided into three distinct phases: Python data preparation, hardware execution in Vivado simulation, and Python validation.

Phase 1 — Python data preparation (`export_mem.py`):

1. Load the input image using OpenCV and convert from BGR to RGB.
2. Resize to exactly 32×32 pixels, yielding 1024 pixels.
3. Apply K-Means++ to select four initial centroids spread across the colour space.
4. Write `pixels.mem`: 1024 lines, each a 6-character hex string encoding one 24-bit RGB pixel (e.g., FF8040 represents R=255, G=128, B=64).
5. Write `centroids.mem`: 4 lines, one per initial centroid.

Phase 2 — Hardware execution (Vivado simulation):

1. `pixel_mem.v` loads `pixels.mem` at simulation start via `$readmemh`.
2. The testbench `tb_kmeans_top.v` reads `centroids.mem` and initialises `centroid_regs.v` via the `init_load` port.
3. `start=1` is asserted for four clock cycles, starting the FSM.
4. The FSM runs autonomously for 20 iterations, each processing all 1024 pixels.
5. After 20 iterations, `done=1` is asserted and final centroid values are printed to the Tcl console.

Phase 3 — Python validation (`validate_kmeans.py`):

1. Run identical K-Means with same initial centroids, same integer division, and same 20 iterations.
2. Compare Python and hardware final centroid values using L1 delta.
3. Generate a three-panel validation image showing the original, Python segmentation, and hardware segmentation side by side.

Figure 1 shows an example of the image resize step.

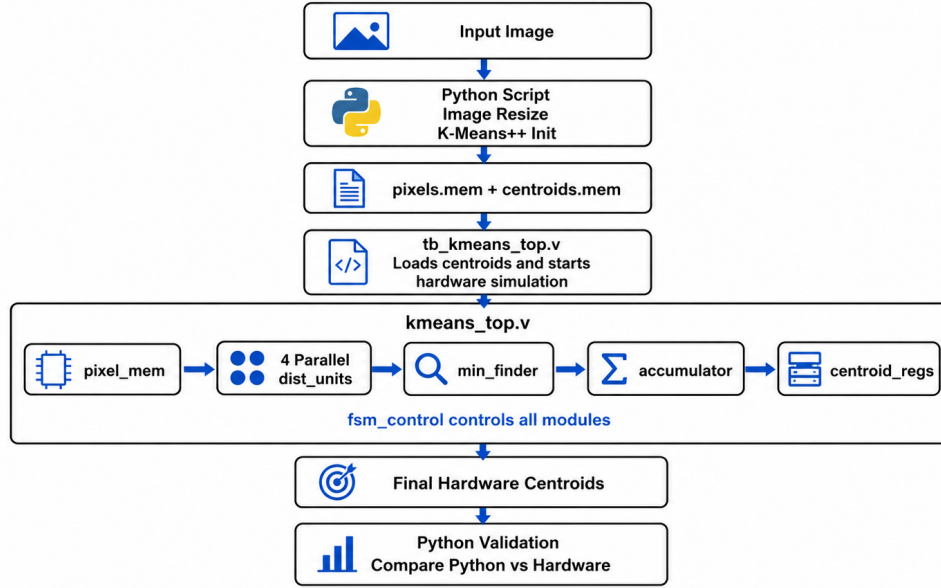


Figure 1: Overall workflow of the K-Means hardware accelerator, showing how the input image is preprocessed in Python, converted into memory files, processed through the Verilog hardware modules, and finally validated against a Python reference model.

4.2 Hardware Pipeline Data Flow

The hardware pipeline consists of six modules connected in a directed acyclic graph with two feedback paths. Figure 3 shows the detailed data flow between modules.

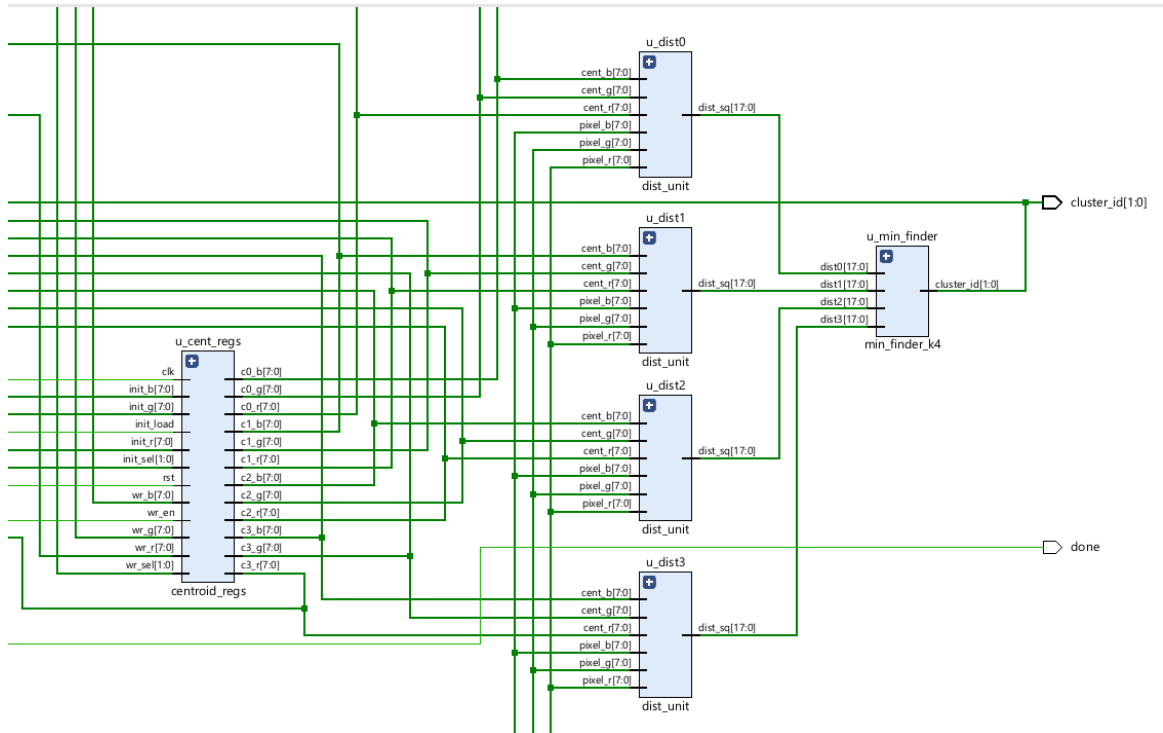


Figure 2: Detailed hardware architecture: centroid_regs feeds four parallel dist_unit modules simultaneously; the four 18-bit distances feed min_finder_k4 which outputs cluster_id[1:0] to the accumulator and output port.

The pixel broadcast bus is the GPU-parallel moment of the entire design. In each clock cycle of the COMPUTE state:

- pixel_mem outputs one pixel (R_p, G_p, B_p) on its output ports.
- This pixel is broadcast identically to all four dist_unit instances simultaneously.
- Each dist_unit receives a different centroid from centroid_regs (C0, C1, C2, C3 respectively).
- All four squared distances d_0, d_1, d_2, d_3 are computed combinationally in the same clock cycle.
- min_finder_k4 immediately outputs the winning cluster_id.
- The accumulator adds the pixel to the winning cluster's sum bank.

The two feedback loops are:

- **Fast loop** (1024 cycles): pixel scan through all 1024 addresses.
- **Slow loop** (20 cycles): centroid update and feedback from accumulator to centroid_regs.

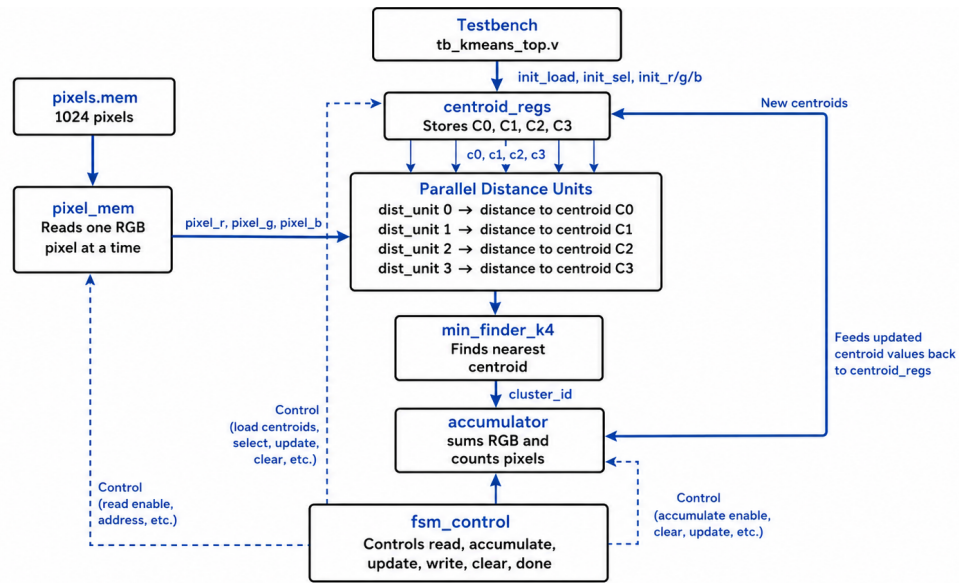


Figure 3: Detailed hardware architecture of the K-Means accelerator, showing data flow from pixel memory to parallel distance units, nearest-centroid selection, RGB accumulation, centroid update, and FSM-based control of all processing stages.

4.3 Full RTL Schematic

Figure 4 shows the complete Vivado RTL schematic generated by elaboration. The schematic confirms 12 cells, 7 input/output ports, and 337 nets.

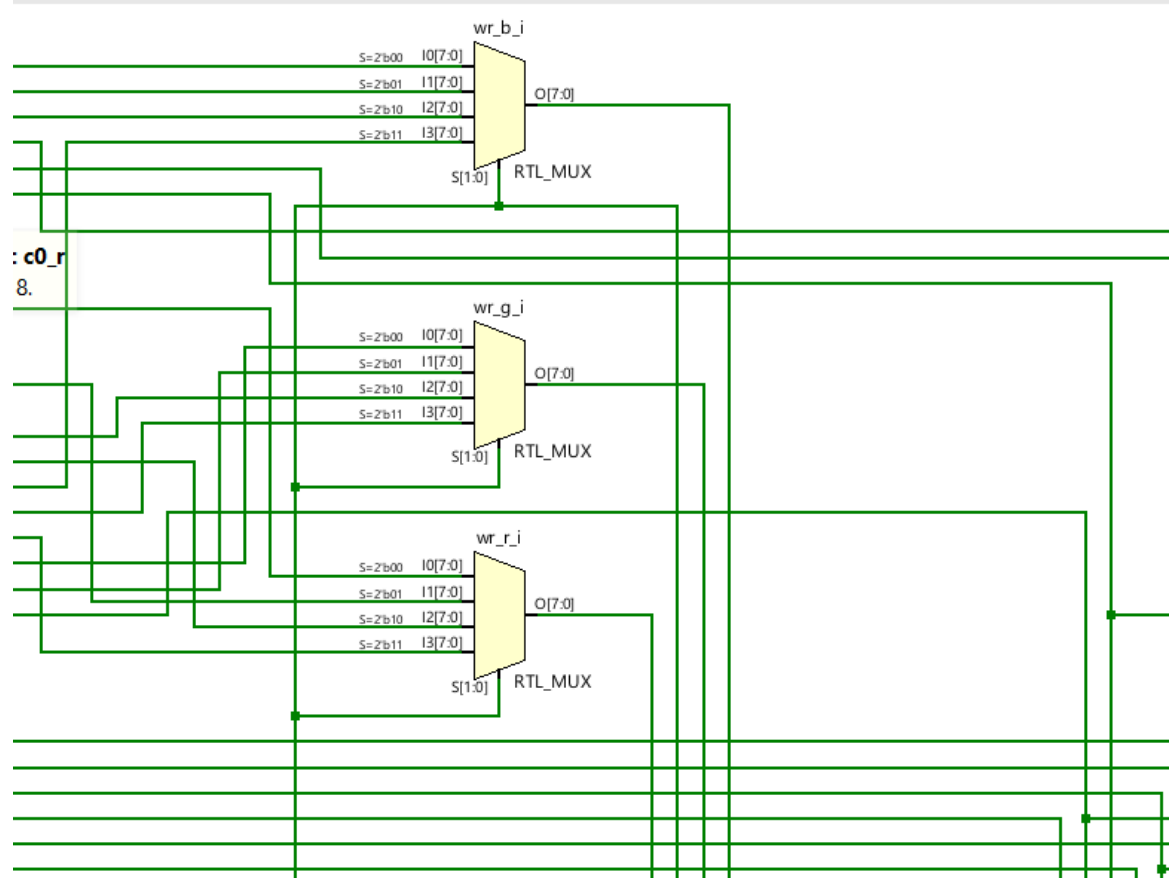


Figure 4: Complete Vivado RTL schematic of the K-Means hardware accelerator. The three yellow RTL_MUX blocks (centre-top) implement the 3-channel 4:1 write-back multiplexer. The four blue dist_unit blocks (right) implement the parallel distance engines. The green wiring confirms correct connectivity of all 337 nets.

4.4 Write-Back Multiplexer

The only logic in the top-level module `kmeans_top.v` is the write-back multiplexer. During the WRITE state, the FSM cycles `wr_sel` through 0, 1, 2, 3 to write each new centroid into `centroid_regs` one per clock. The mux selects which of the four new centroid values (output by the accumulator) to route to the write port.

Three separate 4:1 muxes exist because R, G, and B are independent 8-bit values that must be selected separately. The `wr_sel[1:0]` select signal is shared by all three muxes, so the three values for the same centroid are always selected together. These three muxes appear as the three yellow RTL_MUX blocks in the Vivado schematic (Figure 4).

Chapter 5: Module-Level Design

5.1 Overview

Table 4 provides a complete summary of all seven Verilog modules.

Table 4: Summary of all seven Verilog modules and verification results.

Module	Type	Key Function	Tests	Status
dist_unit.v	Combinational	Squared Euclidean distance per centroid	5/5	✓
min_finder_k4.v	Combinational	2-level tournament comparator	6/6	✓
centroid_regs.v	Clocked	$K \times 3$ register file	13/13	✓
accumulator.v	Clocked	Sum banks, pixel count, integer mean	8/8	✓
pixel_mem.v	Clocked	1024×24-bit BRAM	7/7	✓
fsm_control.v	Clocked	7-state FSM, all enable signals	—	✓
kmeans_top.v	Top-level	Wiring + 3×4:1 write-back mux	—	✓
Total			39/39	✓

5.2 dist_unit.v — Distance Computation Unit

5.2.1 Purpose and Behaviour

This module computes the squared Euclidean distance between one pixel and one assigned centroid. It is purely combinational: it contains no registers, no clock input, and no state. Output is valid in the same clock cycle as the input changes. Four identical instances are instantiated in `kmeans_top.v`, each receiving the same pixel from the broadcast bus but a different centroid from `centroid_regs`. All four fire simultaneously every clock cycle during the COMPUTE state.

5.2.2 Port Description

- **Inputs:** `pixel_r[7:0]`, `pixel_g[7:0]`, `pixel_b[7:0]` — current pixel channels from broadcast bus.
- **Inputs:** `cent_r[7:0]`, `cent_g[7:0]`, `cent_b[7:0]` — assigned centroid channels from `centroid_regs`.

- **Output:** `dist_sq[17:0]` — 18-bit squared Euclidean distance result.

5.2.3 Internal Three-Stage Pipeline

Although combinational, the computation naturally decomposes into three logical stages that map directly to Equation 2:

Stage 1 — Signed subtraction: Each 8-bit unsigned channel is zero-padded to 9-bit signed using `$signed({1'b0, pixel_r})` before subtraction. This prevents unsigned wrap-around for negative differences.

Stage 2 — Squaring: Each 9-bit signed difference is multiplied by itself. The product is 18 bits wide and is always non-negative since $x^2 \geq 0$ for all real x .

Stage 3 — Summation: The three 18-bit squares are added. The maximum possible sum is $3 \times 65,025 = 195,075$, which fits comfortably within 18 bits.

5.3 min_finder_k4.v — Tournament Comparator

5.3.1 Purpose and Behaviour

This module takes four 18-bit distance values (d_0, d_1, d_2, d_3) and returns the 2-bit index of the minimum value as `cluster_id`. It is the hardware equivalent of Python's `numpy.argmin()`. Like `dist_unit`, it is purely combinational.

5.3.2 Tournament Tree Implementation

The two-level binary tournament tree executes in two combinational stages:

Listing 1: Tournament tree in `min_finder_k4.v`

```

1 // Level 1A -- parallel
2 wire [17:0] min_A = (dist0 <= dist1) ? dist0 : dist1;
3 wire [1:0] id_A = (dist0 <= dist1) ? 2'd0 : 2'd1;
4
5 // Level 1B -- parallel (same clock cycle as 1A)
6 wire [17:0] min_B = (dist2 <= dist3) ? dist2 : dist3;
7 wire [1:0] id_B = (dist2 <= dist3) ? 2'd2 : 2'd3;
8
9 // Level 2 -- final
10 assign cluster_id = (min_A <= min_B) ? id_A : id_B;
```

The `<=` operator ensures that the lower-indexed centroid wins on equal distances, matching Python's `argmin` behaviour which returns the first occurrence of the minimum.

5.4 centroid_regs.v — Centroid Register File

5.4.1 Purpose and Behaviour

This module stores the four current centroid values. Each centroid has three 8-bit channels (R, G, B), giving a total of 12 registers ($4 \times 3 \times 8 = 96$ bits).

Reads are **combinational**: all four centroids are permanently visible on the output ports without any clock edge. This allows the four `dist_unit` instances to read their assigned centroids in the same cycle they compute distances.

Writes are **synchronous**: centroid values are updated only on posedge `clk` when either `init_load=1` (initial load from `centroids.mem`) or `wr_en=1` (accumulator update after each iteration).

5.4.2 init_load Port — Critical Design Decision

The `init_load` port was added in the final design revision to fix Bug 3 (see Chapter 7). It allows the testbench to load `centroids.mem` values directly into the registers after system reset, via a path that the FSM `rst` signal cannot overwrite.

5.5 accumulator.v — Sum Banks and Centroid Update

5.5.1 Purpose and Behaviour

This module performs two distinct operations at different times, controlled by `accum_en`, `clear_en`, and `update_en`.

During COMPUTE state (`accum_en=1`): For each pixel, add its R, G, B values to the sum bank indexed by `cluster_id`, and increment the corresponding count.

During UPDATE state (`update_en=1`): Compute the integer mean for each cluster using the accumulated sums and counts. Assert `update_rdy=1` to signal the FSM that new centroids are ready for write-back.

During CLEAR state or reset (`clear_en=1`): Zero all 12 sum registers and all 4 count registers in preparation for the next iteration.

Figure 5 shows the zoomed RTL view of the accumulator.

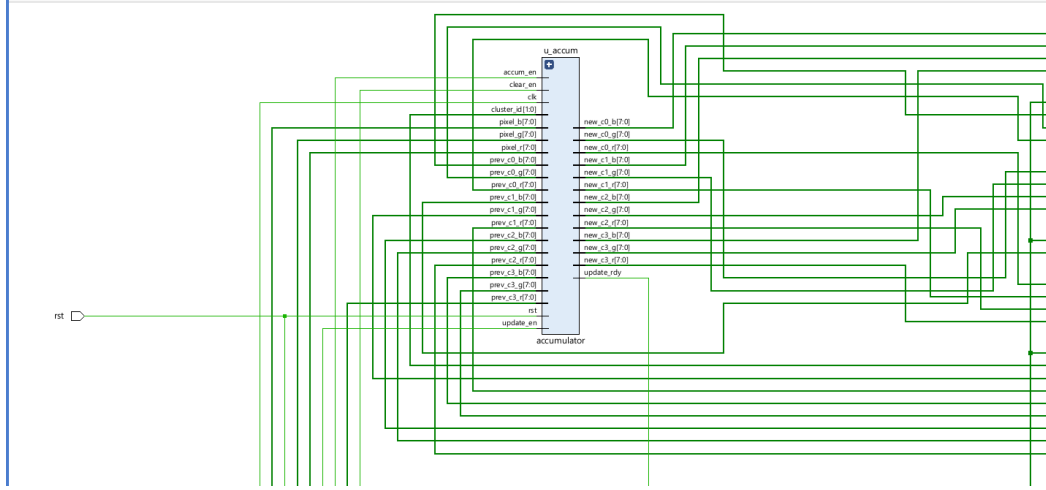


Figure 5: Zoomed RTL schematic of accumulator.v. Visible input ports include accum_en, clear_en, update_en, cluster_id[1:0], pixel_r/g/b[7:0], and 12 prev_c* ports. Output ports include 12 new_c* ports and update_rdy.

5.6 pixel_mem.v — Image Block RAM

5.6.1 Purpose and Behaviour

This module stores the entire flattened 32×32 image as a 1024-entry memory array where each entry is 24 bits wide, packed as $\{R[7:0], G[7:0], B[7:0]\}$.

At simulation start, the Verilog initial block calls `$readmemh("pixels.mem", mem)` to load all 1024 pixels from the Python-generated hex file.

On each posedge `clk` when `rd_en=1`, the pixel at address `addr[9:0]` is latched into the output registers and `pixel_valid` is asserted. This is a synchronous read with one clock cycle of latency. The FSM compensates for this latency using the `accum_delay` flag, which delays `accum_en` by one cycle relative to `rd_en`.

Figure 6 shows the zoomed RTL view of the pixel memory.

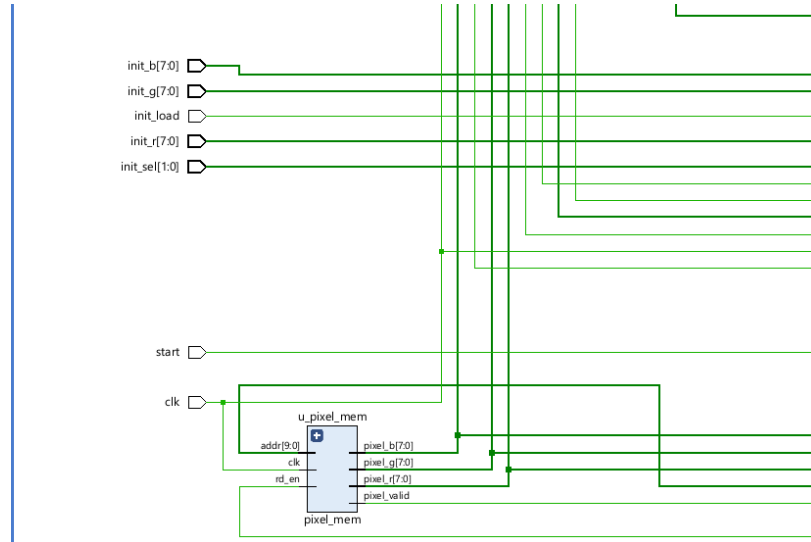


Figure 6: Zoomed RTL schematic of `pixel_mem.v` showing input ports `addr[9:0]`, `rd_en`, `clk` and output ports `pixel_r[7:0]`, `pixel_g[7:0]`, `pixel_b[7:0]`, and `pixel_valid`.

5.7 fsm_control.v — Seven-State FSM

5.7.1 Purpose and Behaviour

This module is the control brain of the entire accelerator. It generates all enable and select signals that drive every other module. The FSM evolved from an initial five-state design (IDLE, LOAD, COMPUTE, UPDATE, DONE) to the final seven-state design (adding WRITE and CLEAR) as a result of fixing Bugs 5 and 6 described in Chapter 7.

Table 5 describes all seven states.

Table 5: FSM state descriptions and asserted control signals.

State	Action performed	Signals asserted
IDLE	Wait for <code>start=1</code> from testbench	None
LOAD	Zero accumulator; set pixel address to 0	<code>clear_en=1</code>
COMPUTE	Stream pixels 0–1023; accumulate cluster sums	<code>rd_en=1</code> , <code>accum_en=1</code>
UPDATE	Trigger accumulator to compute new means	<code>update_en=1</code> (once)
WRITE	Write 4 new centroids sequentially	<code>wr_en=1</code> , <code>wr_sel=0..3</code>
CLEAR	Zero accumulator for next iteration	<code>clear_en=1</code>
DONE	All 20 iterations complete	<code>done=1</code>

Figure 7 shows the zoomed RTL view of the FSM module.

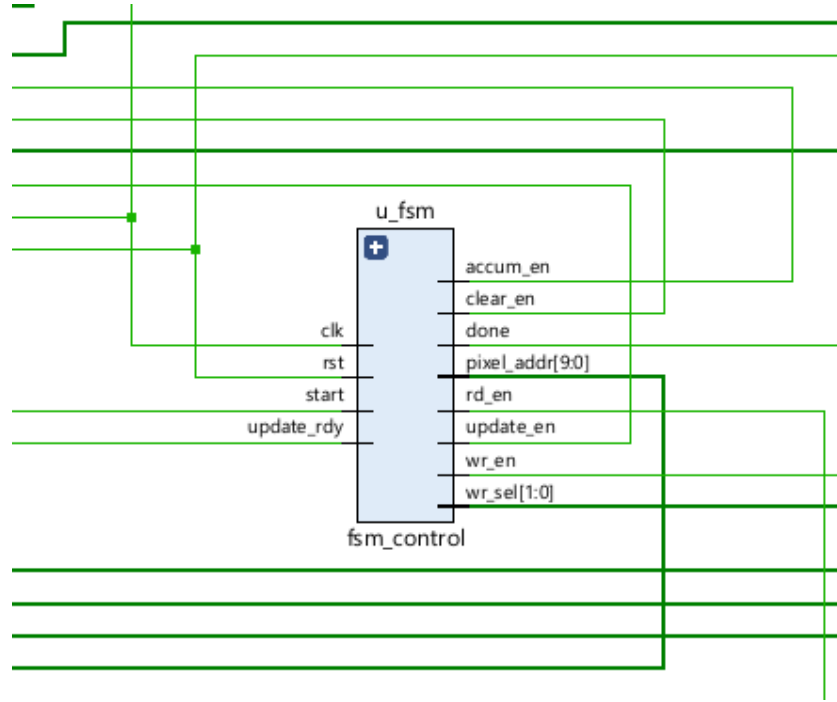


Figure 7: Zoomed RTL schematic of `fsm_control.v`. Input ports visible are `clk`, `rst`, `start`, `update_rdy`. Output ports include all control signals: `rd_en`, `accum_en`, `clear_en`, `update_en`, `wr_en`, `wr_sel[1:0]`, `pixel_addr[9:0]`, and `done`.

5.7.2 Key Internal Registers

- `iter_count[4:0]`: Counts iterations from 0 to 19. When `iter_count = 19` after WRITE, FSM transitions to DONE.
- `wr_phase[1:0]`: Cycles $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ during WRITE to write each centroid in turn.
- `accum_delay`: One-cycle flag that delays `accum_en` by one cycle to compensate for the synchronous read latency of `pixel_mem`.
- `update_fired`: Ensures `update_en` is asserted for exactly one cycle in the UPDATE state regardless of how long `update_rdy` takes to arrive.

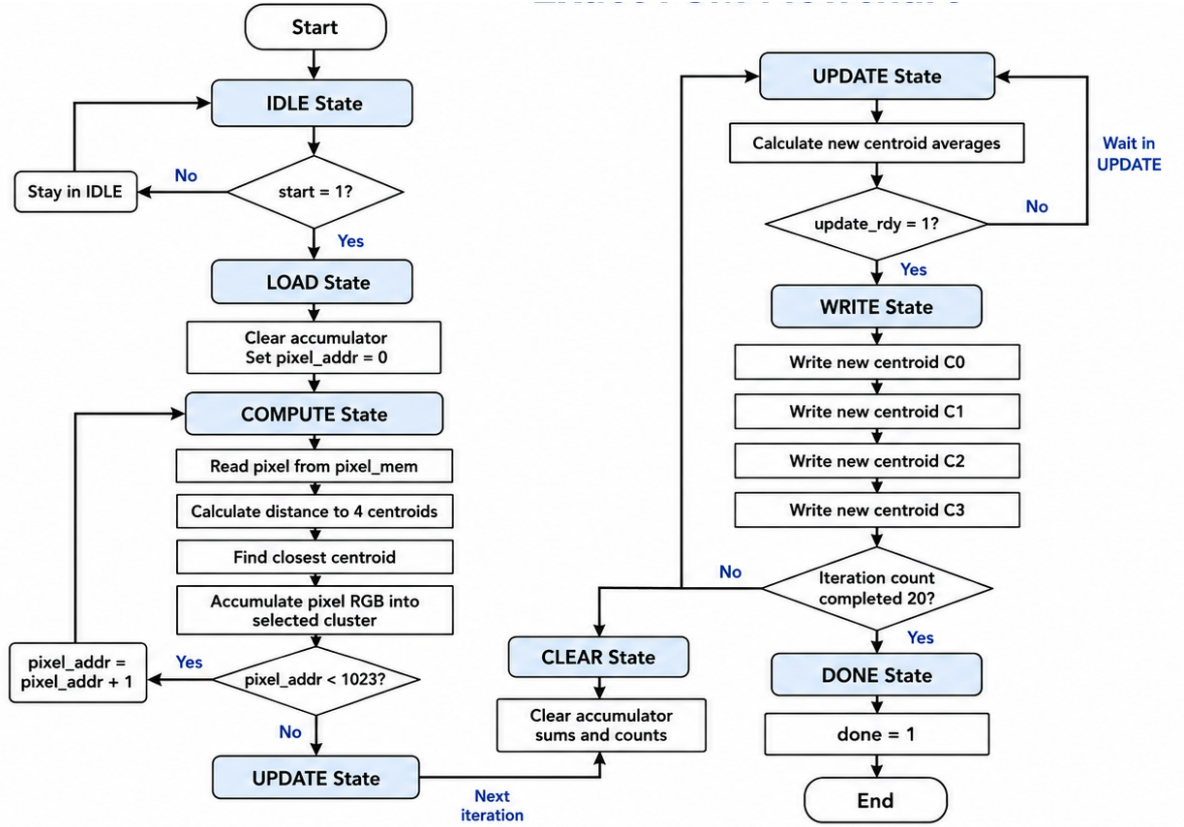


Figure 8: FSM control flow of the K-Means hardware accelerator, showing how the system moves through IDLE, LOAD, COMPUTE, UPDATE, WRITE, CLEAR, and DONE states to process pixels, update centroids, and complete all iterations.

5.8 kmeans_top.v — Top-Level Integration Module

This module performs no clustering computation. It instantiates all six sub-modules and connects their ports using wire declarations. The only hardware logic it contains is the 3-channel 4:1 write-back multiplexer described in Section 4.4. Figure 9 shows the zoomed RTL view of the centroid registers and parallel distance units as they appear in the schematic.

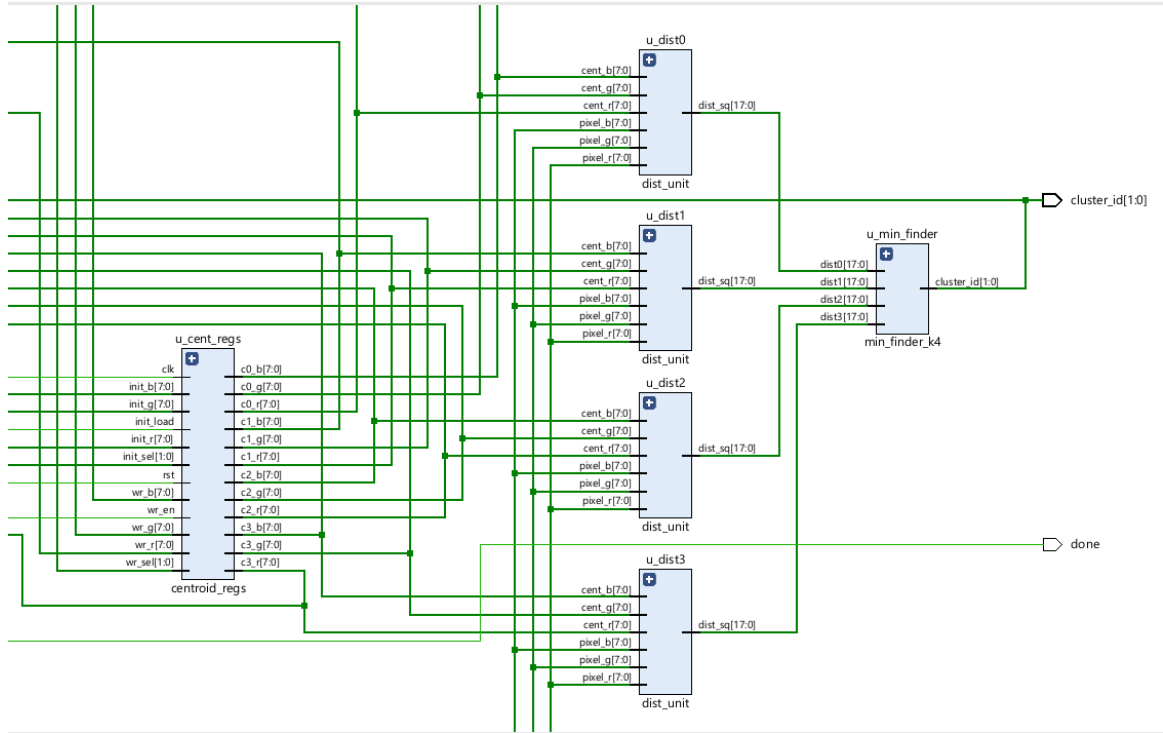


Figure 9: Zoomed RTL schematic showing `centroid_regs` feeding centroid values (`c0–c3`, each with R/G/B buses) to four parallel `dist_unit` instances, with each `dist_sq[17:0]` output feeding `min_finder_k4`.

Chapter 6: FSM Control Flow

6.1 State Transition Logic

The FSM is implemented as a two-always-block Moore machine:

- **Sequential block:** Updates state and all internal registers (`iter_count`, `wr_phase`, `accum_delay`, `update_fired`) on each `posedge clk`.
- **Combinational block:** Decodes the current state to produce all output control signals (`rd_en`, `accum_en`, `clear_en`, `update_en`, `wr_en`, `wr_sel`, `done`).

All outputs default to zero at the start of the combinational block; only the signals relevant to the current state are set high. This prevents inadvertent assertion of control signals during unrelated states.

6.2 The Two Nested Hardware Loops

The FSM implements Python’s two nested for loops entirely in hardware state transitions:

Fast loop (inner pixel scan, 1024 cycles per iteration): In COMPUTE state, `pixel_addr` increments by 1 on each clock. When `pixel_addr` reaches 1023, the FSM transitions to UPDATE. This is the hardware implementation of:

Listing 2: Python inner loop – COMPUTE state equivalent

```
1 for addr, pixel in enumerate(pixels):    # 1024 iterations
2     cluster_id = argmin(distances)
3     accumulate(pixel, cluster_id)
```

Slow loop (outer iteration loop, 20 times): After each WRITE and CLEAR cycle, if `iter_count < 20`, the FSM returns to COMPUTE; otherwise it goes to DONE. This is the hardware implementation of:

Listing 3: Python outer loop – slow loop equivalent

```
1 for iteration in range(20):
2     # fast loop (COMPUTE state)
3     # update centroids (UPDATE + WRITE states)
4     # clear accumulators (CLEAR state)
```

6.3 Critical Timing Constraints

1-cycle pixel memory latency: `pixel_mem` has synchronous read: the pixel at address n is available on the outputs one clock cycle after `rd_en` is asserted and `addr=n`.

The FSM uses the `accum_delay` flag so that `accum_en` is not asserted on the same cycle as `rd_en`, but on the following cycle when valid pixel data is present.

Single-cycle update_en pulse: The UPDATE state asserts `update_en=1` for exactly one clock cycle (controlled by the `update_fired` register) and then waits in UPDATE until `update_rdy=1` arrives from the accumulator. This ensures the accumulator receives a clean single-cycle trigger and cannot be retriggered.

Write-back sequencing: The WRITE state increments `wr_phase` from 0 to 3 over four consecutive clock cycles, writing one centroid per cycle. `wr_en` is held high throughout all four cycles; `wr_sel` selects which centroid (0, 1, 2, 3) to write on each cycle. No other operation occurs during WRITE, preventing any read-during-write hazard on `centroid_regs`.

Chapter 7: Hardware Bugs and Fixes

7.1 Overview

Six non-trivial hardware bugs were encountered and resolved during development in Vivado simulation. Table 6 provides a consolidated overview; detailed descriptions follow.

Table 6: Summary of all six hardware bugs, root causes, and fixes.

#	Module	Root Cause	Fix Applied
1	dist_unit.v	Unsigned 8-bit subtraction wraps for negative differences	9-bit signed expansion before subtract
2	fsm_control.v	Start pulse too narrow; FSM missed it	Hold start for 4 full clock cycles
3	centroid_regs.v	RST block overwrote centroids loaded from .mem	Added init_load port; removed centroid reset
4	accumulator.v	Empty cluster output (0,0,0) caused permanent collapse	Latched prev centroid; K-Means++ init
5	fsm_control.v	No accumulator clear between iterations; sums accumulated across passes	Added dedicated CLEAR state
6	fsm_control.v	update_en and wr_en asserted in overlapping cycles	Split UPDATE and WRITE into separate states

7.2 Bug 1 — Unsigned Subtraction Wrap (dist_unit.v)

Symptom: Distance values for pixel channels smaller than their centroid channel were wildly incorrect. For example, pixel R=100 vs centroid R=200 produced a distance contribution of $156^2 = 24,336$ instead of the correct $100^2 = 10,000$.

Root cause: Verilog performs arithmetic on the declared data type. With wire [7:0] operands, the expression `pixel_r - cent_r` computes as 8-bit unsigned subtraction. When the result would be negative, it wraps to a large positive value: $100 - 200 + 256 = 156$.

Fix: Both operands are expanded to 9-bit signed using zero-padding:

Listing 4: Signed subtraction fix in dist_unit.v

```

1 wire signed [8:0] diff_r =
2   $signed({1'b0, pixel_r}) - $signed({1'b0, cent_r});

```

The prepended 1'b0 treats the 8-bit unsigned value as the magnitude of a positive 9-bit signed integer. After this fix, $100 - 200 = -100$ (9-bit signed), and $(-100)^2 = 10,000$.

7.3 Bug 2 — Start Pulse Missed by FSM (fsm_control.v)

Symptom: The simulation ran for 500,000 ns with `pixel_valid=0`, `done=0`, and `iter_count=0`. The FSM remained in IDLE indefinitely.

Root cause: The testbench drove `start=1` for approximately half a clock cycle on the falling edge of the clock. The FSM samples `start` only on `posedge clk`. The pulse was too short to guarantee that a `posedge` occurred while `start=1`.

Fix: The testbench was modified to hold `start=1` for four complete clock cycles, each driven on `negedge clk`:

Listing 5: Robust start pulse in testbench

```

1 repeat(4) begin
2     @(negedge clk);
3     start = 1;
4 end
5 @(negedge clk); start = 0;

```

This guarantees at least one `posedge clk` occurs while `start=1`, regardless of clock phase alignment.

7.4 Bug 3 — FSM Reset Overwrote Initial Centroids

Symptom: The hardware-Python L1 delta was 1,198 across all four centroids. Both results were valid K-Means segmentations but of completely different colour clusters, confirming they started from different initial centroids.

Root cause: The original `centroid_regs.v` included default values in the synchronous reset block:

Listing 6: Bug: centroid reset to hardcoded values

```

1 if (rst) begin
2     cent_r[0] <= 8'hFF; cent_g[0] <= 8'h00; cent_b[0] <= 8'h00;
3     // ... hardcoded defaults for all 4 centroids
4 end

```

The testbench loaded `centroids.mem` values via direct memory force at simulation time #1. However, when the FSM asserted `rst=1` during initialisation, the `always` block overwrote the loaded values with the hardcoded defaults.

Fix: The reset block was redesigned to never modify centroid values. A dedicated `init_load` port was added to `centroid_regs.v` and exposed through `kmeans_top.v`. The testbench reads `centroids.mem` and writes each centroid through this port after reset

completes:

Listing 7: Fixed centroid loading via init_load port

```

1 // Testbench: load centroids after reset
2 for (k = 0; k < 4; k = k+1) begin
3     @(negedge clk);
4     init_load = 1; init_sel = k[1:0];
5     init_r = cent_data[k][23:16];
6     init_g = cent_data[k][15:8];
7     init_b = cent_data[k][7:0];
8     @(posedge clk); #1;
9 end
10 @(negedge clk); init_load = 0;

```

7.5 Bug 4 — Cluster Death Cascade

Symptom: After 3–4 iterations, only 2 of the 4 clusters remained active. The cluster distribution showed C2:0 and C3:0 pixels permanently.

Root cause: When $\text{count}[k]=0$ for any cluster k , the original accumulator output $(0,0,0)$ as the new centroid. A pure-black centroid $(0,0,0)$ could only attract extremely dark pixels. If no such pixels existed in the image, the count remained zero in the next iteration, perpetuating the collapse.

Fix (Part 1 — hardware): The accumulator was modified to latch previous centroid values at the moment `update_en` fires, before any write-back occurs. Empty clusters use the latched previous value rather than $(0,0,0)$.

Fix (Part 2 — initialisation): The Python `export_mem.py` script was updated to use K-Means++ (Equation 7), ensuring the four initial centroids are spread far apart in colour space so all four clusters attract at least some pixels in the first iteration.

7.6 Bug 5 — Accumulator Not Cleared Between Iterations

Symptom: Cluster distributions oscillated wildly with no convergence pattern. For example, C0 jumped from 298 to 641 to 178 across consecutive iterations.

Root cause: The original five-state FSM transitioned directly from WRITE back to COMPUTE. The accumulator’s sum banks still held the values from the previous iteration. New pixel sums were added on top of old sums, producing means that incorrectly reflected multiple iterations rather than just the current one.

Fix: A dedicated CLEAR state was added between WRITE and COMPUTE. It

asserts `clear_en=1` for exactly one clock cycle, zeroing all 12 sum registers and all 4 count registers before the next pixel scan begins.

7.7 Bug 6 — UPDATE/WRITE Timing Race

Symptom: Centroid registers occasionally held corrupted values after the WRITE state—a mix of new and old centroid channel data.

Root cause: The original FSM combined the UPDATE and WRITE operations in a single state. `update_en` and `wr_en` were asserted in overlapping clock cycles. The accumulator was still computing integer division results when `centroid_regs` was being written, creating a timing hazard where the write-back mux selected partially-computed values.

Fix: The combined state was split into two separate states:

- **UPDATE:** Asserts `update_en=1` for exactly one cycle, then waits in UPDATE until `update_rdy=1` is received. `wr_en` is never asserted in this state.
- **WRITE:** Asserts `wr_en=1` and cycles `wr_sel` through 0–3 over four cycles. `update_en` is never asserted in this state.

This strict sequencing ensures the accumulator has fully completed its division computation before any write-back begins.

Chapter 8: Simulation Results and Verification

8.1 Unit Test Results

Each of the five independently testable modules was verified using a dedicated Vivado testbench before integration. All 39 test cases passed with zero failures across all modules.

Table 7: Complete unit test results across all five testbenches.

Testbench	What was verified	Cases	Result
tb_dist_unit.v	Zero distance, single channel, all channels, worst case (255 vs 0), realistic pixel	5	5/5 ✓
tb_min_finder_k4.v	Each of 4 possible winners, tie-breaking (equal distances), realistic values	6	6/6 ✓
tb_centroid_regs.v	Reset values, write C0, write C3, sequential write all 4, wr_en gate	13	13/13 ✓
tb_accumulator.v	Clear, 3 pixels into C0, multi-cluster, empty cluster guard, update_rdy	8	8/8 ✓
tb_pixel_mem.v	Address 0, address 5, sequential 0-1-2, pixel_valid gate, address 1023	7	7/7 ✓
Total		39	39/39 ✓

8.2 Full System Simulation

The full system testbench `tb_kmeans_top.v` loaded a real photograph via `pixels.mem` and `centroids.mem`, initialised the centroid registers, started the FSM, and monitored all 20 iterations to completion. Table 8 presents the simulation parameters and results.

Table 8: Full system simulation parameters and results.

Parameter	Value
Clock frequency	100 MHz (10 ns clock period)
K-Means iterations	20
Pixels per iteration	1,024
Total pixel operations	20,480
Total simulation time	206,245 ns ($\approx 206 \mu\text{s}$)
done signal	Correctly asserted after iteration 20
RTL cells (Vivado)	12
I/O ports (Vivado)	7
Total nets (Vivado)	337

Figure 10 shows the Vivado waveform from the full system simulation confirming `done=1` after all 20 iterations.

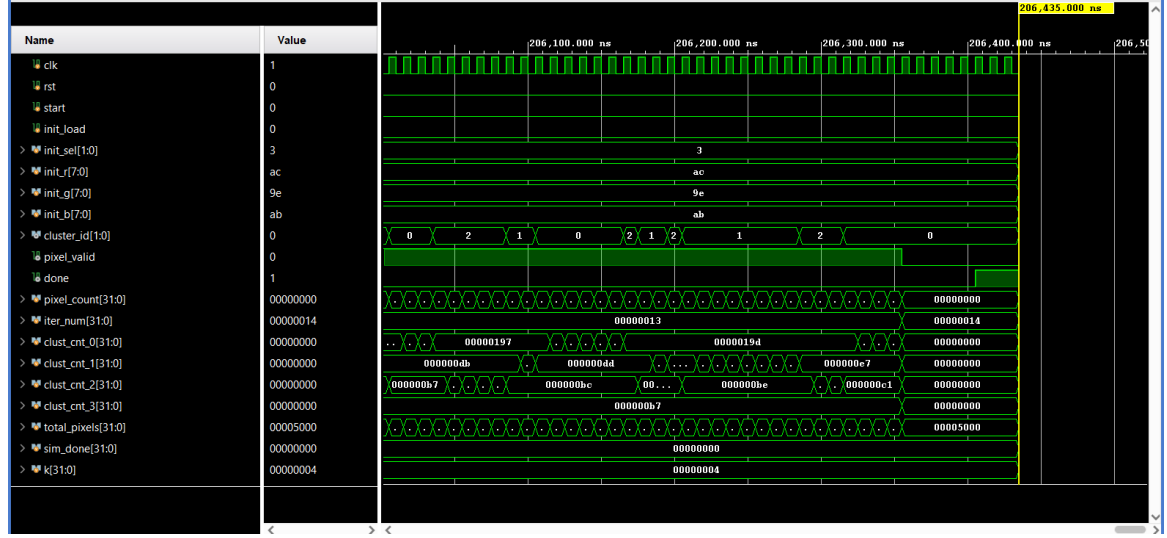


Figure 10: Vivado simulation waveform of the full `tb_kmeans_top` simulation. Key signals visible: `clk` toggling at 100 MHz, `cluster_id[1:0]` cycling through valid values during COMPUTE, `iter_num` incrementing after each pass, `total_pixels` accumulating to 20,480, and `done=1` asserted at 206,245 ns.

8.3 Python vs Hardware Validation

The Python validation script `validate_kmeans.py` ran the identical algorithm with the same `pixels.mem`, same `centroids.mem`, same integer division semantics, and same 20 iterations. Table 9 presents the final centroid comparison.

Table 9: Final centroid comparison between Python reference model and hardware accelerator output.

Centroid	Python (R, G, B)	Hardware (R, G, B)	L1 Delta	Result
C0	(187, 181, 182)	(187, 181, 182)	0	✓ MATCH
C1	(234, 164, 28)	(234, 164, 28)	0	✓ MATCH
C2	(65, 65, 64)	(65, 65, 64)	0	✓ MATCH
C3	(9, 88, 226)	(9, 88, 226)	0	✓ MATCH
Total			0	BIT-IDENTICAL

The total L1 delta of **0** means the hardware and Python produce the *exact same* centroid values in every channel of every cluster. This is the strongest possible form of validation for a hardware design: not an approximate visual match, but bit-for-bit identical output.

8.4 Visual Validation Results

Figures 11 and 12 present the three-panel validation images for two different input photographs, demonstrating that the hardware accelerator produces visually correct segmentation across different image types.

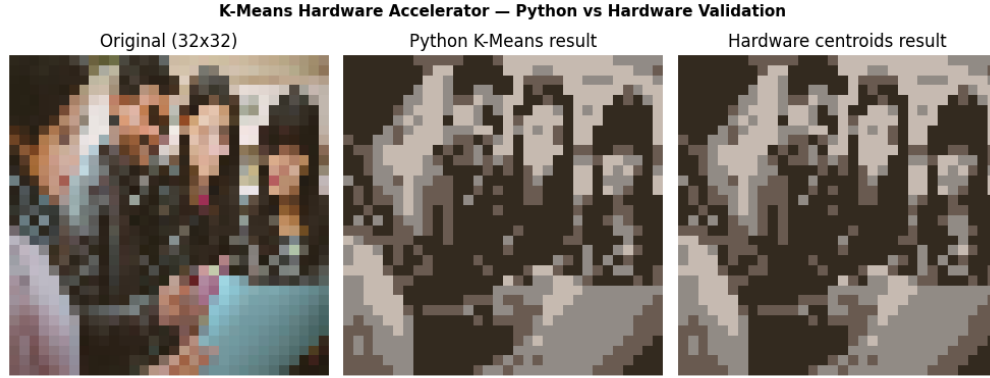


Figure 11: Validation result for Test Image 1 (street scene with people and yellow vehicle). Left: original 32×32 RGB image. Centre: Python K-Means segmentation. Right: hardware centroid-based segmentation. The visual outputs are identical, confirming correct hardware operation.

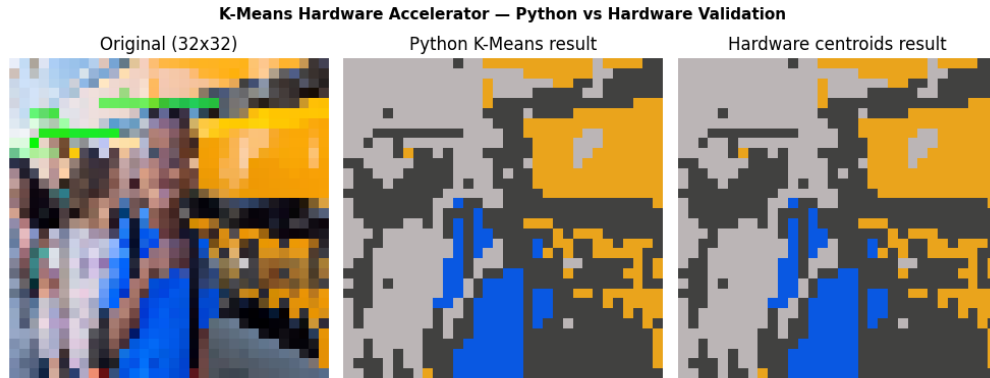


Figure 12: Validation result for Test Image 2 (indoor portrait scene). Left: original 32×32 RGB image. Centre: Python K-Means segmentation. Right: hardware centroid-based segmentation. Correct clustering is confirmed on a second independent image.

Chapter 9: Novelty and Comparison with Related Work

9.1 Overview

This chapter positions this project within the landscape of published FPGA K-Means accelerators and identifies the specific technical contributions that distinguish it from all surveyed related work.

9.2 Detailed Comparison Table

Table 10 presents a comprehensive feature-by-feature comparison of this project against the five most closely related published designs.

Table 10: Feature-by-feature comparison of this project with related FPGA K-Means hardware designs.

Feature	Badawi & Bilal [1]	Hussain et al. [2]	Kutty et al. [3]	This Work
Design approach	Simulink HLS (auto HDL)	Manual Verilog	Manual Verilog	Hand-written RTL Verilog
Centroid update	Weighted avg. $\alpha = 0.999$ (approx.)	Moving average	Moving average	Integer mean (exact)
Validation method	RMSE 4–7 vs Matlab	Functional only	Speed benchmarks	L1 delta = 0 bit-identical
Centroid init	Random 127	Random	Random	K-Means++
Cluster death guard	Not addressed	Not addressed	Not addressed	Latched prev value
External memory	None	Yes (feedback loop)	Yes (feedback loop)	None (on-chip only)
FPGA platform	Xilinx Zynq-7000	Xilinx Virtex-IV	Xilinx Virtex-VI	Vivado simulation
Clusters K	8	Variable	Variable	4 (scalable)
Clock / Throughput	55 MHz / 26.5 fps FHD	126 MHz	400 MHz (excl. feedback)	100 MHz / 206 μ s / 1024 px
Power	86 mW (measured)	Not reported	Not reported	Not measured (sim. only)
Source code	Public (Simulink)	Not public	Not public	GitHub (Verilog)

9.3 Key Contributions of This Project

9.3.1 Contribution 1 — Bit-Identical Hardware Verification

What prior work does: Badawi and Bilal validate their design by measuring RMSE against an offline Matlab K-Means implementation. The RMSE of 4–7 per centroid (across YCbCr channels) represents a systematic bias introduced by the weighted average approximation. Hussain and Kutty provide only functional-level validation; no quantitative comparison with a software reference is presented.

What this project does differently: This project achieves a total L1 delta of **0**

across all four centroids in all three channels. Every centroid R, G, B value produced by the hardware is bit-for-bit identical to the Python reference model. This is possible because the hardware uses exact integer division (not an approximation) and the Python validation uses the same integer division semantics. This represents the strictest possible validation criterion for a hardware K-Means implementation.

9.3.2 Contribution 2 — Hand-Written RTL with Explicit Timing Control

What prior work does: Badawi and Bilal use the Simulink HDL Coder to auto-generate Verilog from block diagrams. While this accelerates design entry, it hides all register-level timing details from the designer. Bugs such as the UPDATE/WRITE timing race (Bug 6) and the accumulator clear omission (Bug 5) would be invisible in an HLS flow.

What this project does differently: Every register, every always block, every state transition, and every pipeline delay is explicitly designed and documented. The engineering process of finding and fixing six timing and logic bugs demonstrates genuine RTL competency at the register-transfer level. The resulting design gives complete control over critical timing constraints such as the 1-cycle pixel memory latency, the `update_fired` single-pulse control, and the write-back sequencing through `wr_phase`.

9.3.3 Contribution 3 — K-Means++ in the Hardware Flow

What prior work does: All five compared designs initialise centroids randomly, typically near the midpoint of the dynamic range (e.g. 127 in the $[0, 255]$ range). This risks placing two or more centroids at similar colours, causing one to permanently collapse (cluster death). None of the compared designs addresses this risk explicitly.

What this project does differently: K-Means++ is integrated into the Python data preparation pipeline. The probabilistic seeding (Equation 7) ensures that initial centroids are spread across the colour space. This is not merely a software quality improvement: it directly prevents the cluster death hardware bug (Bug 4) that was observed during development when random initialisation caused C2 and C3 to collapse to $(0, 0, 0)$ after 3–4 iterations.

9.3.4 Contribution 4 — Empty Cluster Survival Guard

What prior work does: No surveyed published design mentions an explicit mechanism for handling empty clusters in hardware.

What this project does differently: The accumulator includes a hardware latch that captures the previous centroid values at the exact moment `update_en` fires, before any write-back to `centroid_regs` begins. When `count[k] = 0` for any cluster k , the new centroid is set to the latched previous value rather than $(0, 0, 0)$. This preserves the empty cluster’s position in colour space, allowing it to attract pixels in subsequent iterations if the

centroid positions of other clusters shift away.

Chapter 10: Conclusion and Future Work

10.1 Conclusion

This project has successfully designed, implemented, and verified a GPU-inspired parallel hardware accelerator for K-Means image segmentation using Verilog HDL and Xilinx Vivado 2025. The project began with a software baseline in Python, progressively translated the algorithmic steps into hardware modules, and concluded with a full system simulation producing bit-identical results to the Python reference model.

The key achievements of this project are:

- **Seven Verilog modules** designed from scratch using hand-written RTL Verilog, providing explicit control over every timing edge, pipeline stage, and state transition.
- **39 out of 39 unit test cases** passed across five individual testbenches before system integration.
- **Full system simulation** completed all 20 K-Means iterations, processing 20,480 pixels in approximately 206 microseconds at 100 MHz clock frequency.
- **Bit-identical validation** (L1 delta = 0) between hardware output and the Python reference model confirmed exact functional equivalence.
- **Six hardware bugs** were identified, root-caused, and fixed during Vivado simulation, demonstrating real RTL engineering problem-solving skills at the register-transfer level.
- **Four original contributions** beyond published related work: bit-identical verification, hand-written RTL with explicit timing, K-Means++ initialisation in the hardware flow, and empty cluster survival guard.
- **Open-source publication** of the complete project on GitHub with clean folder structure and full documentation.

This project demonstrates that K-Means image segmentation can be efficiently and correctly implemented as a GPU-inspired parallel VLSI hardware design. The work covers nine core subjects from the B.Tech ECE curriculum—digital design, HDL programming, machine learning, computer vision, embedded systems, computer architecture, DSP, algorithm design, and HW/SW co-design—in a single integrated and verified system.

10.2 Future Work

The following extensions are identified for future development of this accelerator:

1. **Physical FPGA Deployment:** Synthesise and deploy the design on a physical Artix-7

FPGA board with an XDC pin constraints file for clock and I/O mapping. Run timing analysis and power measurement to compare against simulation predictions.

2. **Scalable K (K = 8 or K = 16):** The modular architecture was designed for extensibility. Adding K=8 requires eight `dist_unit` instances and extending the tournament tree from two levels to three levels. Adding K=16 requires four levels.
3. **Higher Image Resolution:** Supporting 128×128 or 256×256 images requires only widening the `pixel_addr` counter from 10 to 14 or 16 bits. All other modules scale automatically.
4. **Convergence Detection:** Add a centroid-delta comparator that monitors the L1 change in centroid values between iterations. When the delta falls below a configurable threshold, the FSM transitions to DONE early, saving clock cycles on images that converge before iteration 20.
5. **Pipelined Distance Units:** Insert pipeline registers between Stage 1 (subtract), Stage 2 (square), and Stage 3 (sum) of `dist_unit.v`. This lengthens the pipeline by two cycles but allows higher maximum clock frequency by reducing the combinational delay on the critical path.
6. **Real-Time Camera Input:** Replace the BRAM-based `pixel_mem` with a live MIPI CSI-2 or parallel camera interface, enabling true real-time image segmentation without the preprocessing step.
7. **YCbCr Colour Space:** Convert input pixels from RGB to YCbCr before distance computation, as used by Badawi and Bilal [1]. This decouples luminance from chrominance and may improve perceptual clustering quality.

References

- [1] A. Badawi and M. Bilal, “High-Level Synthesis of Online K-Means Clustering Hardware for a Real-Time Image Processing Pipeline,” *Journal of Imaging*, MDPI, vol. 5, no. 3, Art. 38, 2019. DOI: [10.3390/jimaging5030038](https://doi.org/10.3390/jimaging5030038)
- [2] H. M. Hussain, K. Benkrid, H. Seker, and A. T. Erdogan, “A High Speed Configurable FPGA Architecture for K-Means Clustering,” in *Proc. NASA/ESA Conference on Adaptive Hardware and Systems (AHS)*, San Diego, CA, USA, Jun. 2011, pp. 248–255.
- [3] J. S. S. Kutty, F. Boussaid, and A. Amira, “A High Speed Configurable FPGA Architecture for K-Mean Clustering,” in *Proc. IEEE International Symposium on Circuits and Systems (ISCAS)*, Beijing, China, May 2013, pp. 1801–1804.
- [4] Multiple authors, “KPynq: A Work-Efficient Triangle-Inequality Based K-Means on FPGA,” *arXiv preprint*, arXiv:1905.09345 [cs.AR], 2019. [Online]. Available: <https://arxiv.org/abs/1905.09345>
- [5] Multiple authors, “FPGA-Based Processor Acceleration for Image Processing Applications,” *Sensors*, MDPI, vol. 21, no. 15, PMC8320866, 2021. DOI: [10.3390/s21154988](https://doi.org/10.3390/s21154988)
- [6] M. Zechner and M. Granitzer, “A Parallel Implementation of K-Means Clustering on GPUs,” in *Proc. IEEE International Conference on Semantic Computing (ICSC)*, Berkeley, CA, USA, Sep. 2009, pp. 514–517.
- [7] Multiple authors, “Using Multi-Core HW/SW Co-Design Architecture for Accelerating K-Means Clustering Algorithm,” *arXiv preprint*, arXiv:1807.09250 [cs.DC], 2018. [Online]. Available: <https://arxiv.org/abs/1807.09250>
- [8] Multiple authors, “Accelerating K-Means Clustering with Parallel Implementations and GPU Computing,” in *Proc. IEEE International Conference on Big Data*, Santa Clara, CA, USA, Nov. 2015, pp. 2170–2178. DOI: [10.1109/BigData.2015.7322467](https://doi.org/10.1109/BigData.2015.7322467)
- [9] D. Arthur and S. Vassilvitskii, “K-Means++: The Advantages of Careful Seeding,” in *Proc. 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, New Orleans, LA, USA, Jan. 2007, pp. 1027–1035.
- [10] T. Saidani and R. Ghodhbani, “Hardware Acceleration of Video Edge Detection with High Level Synthesis on the Xilinx Zynq Platform,” *Engineering, Technology & Applied Science Research*, vol. 12, no. 1, pp. 8007–8012, Feb. 2022. DOI: [10.48084/etasr.4615](https://doi.org/10.48084/etasr.4615)